CS 610 Semester 2024–2025-I: Assignment 4

Name: **Vinayak Devvrat**
Roll Number: **241110079**

November 5, 2024

Problem 1

N=512.
Note: The time taken for each of these implementations also includes the time for `cudaMemcpy` from device to host.
Time taken for serial execution is given by T_{seq} = **1055.57 ms**.
The speed ups mentioned in the subsequent portions are taken with respect to T_{seq}.

Part (i)

The naive approach is implemented by the `kernel1()` function. It uses global memory for both input and output grids. The time taken by this implementation = **259.67 ms** and the speed up is **4.06**.

Part (ii)

This approach makes use of shared memory and is implemented by `kernel2()` function. I experiment with block sides from the set $1, 2, 4, 8$. The results are summarized below.

Table 1: Speed Ups for Stencil Pattern (Shared Memory Tiling)

SNo.	**Block Size**	**Time (ms)**	**Speed Up**
1	1	257.41	4.1
2	2	261.4	4.03
3 (*)	4	255.33	4.13
4	8	258.94	4.07

I get similar speed ups for sizes 2, 4 and 8. Though a speed up is good, in each case it is also offset by the `cudaMemcpy` function.

Part (iii)

This is implemented by the `kernel2_unroll()` function. Unrolling does not affect the speed up by any significant value (**4.07**). Although, this function is commented in the code and can be tried.

Part (iv)

I implement pinned memory using `cudaHostAlloc()`. Time taken in this case is **142.9 ms**, the highest among all implementations. I therefore get a speed up of **7.38**.

Part (v)

I implement UVM using `cudaMallocManaged()`. This gives the worst case speed up. This may be attributed to the frequent references to CPU memory, given a large grid size. There have been cases where the speed up has been worst than serial implementation (as ssen in the logs) and some times marginally better (screenshot attached).

Observation

It has been observed that if I do not include the time taken for `cudaMemcpy` in parts (i) to (iv), I am able to achieve significant speed up (nearly 15-16 times). In essence, it can be said that the computations take very less time but this is heavily offset by memory transfers.
Compilation Command: `nvcc -O2 -std=c++17 -allow-unsupported-compiler -arch=sm_80 -lineinfo -res-usage -src-in-ptx problem1.cu -o prob1.out`
Run Command: `nsys nvprof ./prob1.out`
Logs and a screenshot have been attached as a reference.

Problem 2

Approach:

- To perform exclusive sum in parallel, a binary tree is formed.
- In the Up-sIep phase (as indicated in the code), the partial sums are accumulated in a binary tree and in the Down-sIep phase (as indicated in the code), the exclusive prefix sum is calculated by propagating the partial results down the tree. A final addition takes place which adds results from the parallel computations to the final output.
- Multiple kernels are used to help with addition and handle use cases when size of the array is greater than the block size and/or odd length of N.

A thrust implementation of exclusive prefix sum is also included for comparison.
Speed up is calculated as follows (for N = 2^{16}):
Time taken by Thrust implementation = T_{Thrust} = **7.406 ms**.
Time taken by CUDA implementation = T_{CUDA} = **1.505 ms**.
Speed up = $\frac{T_{Thrust}}{T_{CUDA}} = \frac{7.406}{1.505}$ = **4.92**.
Compilation Command: `nvcc -O2 -std=c++17 -allow-unsupported-compiler -arch=sm_80 -lineinfo -res-usage -src-in-ptx problem2.cu -o prob2.out`
Run Command: `./prob2.out`
A screenshot has been attached as a reference.

Problem 3

Time taken by the sequential implementation (non parallel optimized version of Problem 3) = T_{seq} = **144.94 ms**. All speed ups in the subsequent portions are calculated with respect to T_{seq}.

Part 1

I implemented the CUDA kernel over the non parallel optimized version of the code given for my Assignment 3 submission. Global memory is used in this case.
Time taken by naive implementation = **54.45 ms**.
Speed up acheived = $\frac{144.94}{54.45}$ = **2.66**.
Compilation Command: `nvcc -O3 -std=c++17 -allow-unsupported-compiler -arch=sm_80 -lineinfo -res-usage -src-in-ptx problem3-v0.cu -o prob3.out`
Run Command: `./prob3.out`

Part 2

I have used data prefetching (pre-calculating the necessary multipliers) and `cudaMemAdvise` (to manage memory access for the multipliers) and `cudaMemcpyAsync` for overlapping of computation and memory transfers.
Time taken by naive implementation = **59.68 ms**.
Speed up acheived = $\frac{144.94}{54.45}$ = **2.42**.
Compilation Command: `nvcc -O3 -std=c++17 -allow-unsupported-compiler -arch=sm_80 -lineinfo -res-usage -src-in-ptx problem3-opt.cu -o prob3.out`
Run Command: `./prob3.out`

Part 3

I have implemented this version using `cudaMallocManaged()`.
Time taken by naive implementation = **102.47 ms**.
Speed up acheived = $\frac{144.94}{102.47}$ = **1.41**.
Compilation Command: `nvcc -O3 -std=c++17 -allow-unsupported-compiler -arch=sm_80 -lineinfo -res-usage -src-in-ptx problem3-uvm.cu -o prob3.out`
Run Command: `./prob3.out`

Part 4

I implemented this version by creating the necessary `thrust::device_vector` and `thrust::host_vectors`. Additionally, I also used `thrust::for_each` to parallely compute the points over the entire 13^{10} iteration space.
Time taken by naive implementation = **59.68 ms**.
Speed up acheived = $\frac{144.94}{11.97}$ = **12.10**.
Compilation Command: `nvcc -O3 -std=c++17 -allow-unsupported-compiler -arch=sm_80 -lineinfo -res-usage -src-in-ptx problem3-thrust.cu -o prob3.out`
Run Command: `./prob3.out`

Conclusion

Based on the speed ups archived on the different implementations, it can be concluded that Thrust implementation gave the best speed up for this problem.
Screenshots for each of these cases have been attached as references.

Problem 4

(i) 2D Convolution

For the 2D convolution, speed up is calculated based on the following specifications:

- Size of input matrix = **N** = **512**.
- Size of convolution filter = **7**.

Case 1. Using Global Memory
Time taken by serial implementation = **12.82 ms**.
Time taken by global memory kernel implementation = **2.07 ms**.
Speed up achieved = $\frac{12.82}{2.07}$ = **6.19**.

Compilation Command: `nvcc -O2 -std=c++17 -allow-unsupported-compiler -arch=sm_80 -lineinfo -res-usage -src-in-ptx problem4_2d_global.cu -o prob4.out`
Run Command: `./prob4.out`

Case 2. Using Shared Memory
Time taken by serial implementation = **13.91 ms**.
Time taken by global memory kernel implementation = **1.85 ms**.
Speed up achieved = $\frac{13.91}{7.52}$ = **7.52**.

Compilation Command: `nvcc -O2 -std=c++17 -allow-unsupported-compiler -arch=sm_80 -lineinfo -res-usage -src-in-ptx problem4_2d_shared.cu -o prob4.out`
Run Command: `./prob4.out`

(ii) 3D Convolution

For the 3D convolution, speed up is calculated based on the following specifications:

- Size of input matrix = **N** = **128**.
- Size of convolution filter = **11**.

Case 1. Using Global Memory
Time taken by serial implementation = **3563.38 ms**.
Time taken by global memory kernel implementation = **33.027 ms**.
Speed up achieved = $\frac{3563.38}{33.037}$ = **107.9**.

Compilation Command: `nvcc -O2 -std=c++17 -allow-unsupported-compiler -arch=sm_80 -lineinfo -res-usage -src-in-ptx problem4_3d_global.cu -o prob4.out`
Run Command: `./prob4.out`

Case 2. Using Shared Memory
Time taken by serial implementation = **3656.36 ms**.
Time taken by global memory kernel implementation = **44.29 ms**.
Speed up achieved = $\frac{3656.36}{44.29}$ = **82.55**.

Compilation Command: `nvcc -O2 -std=c++17 -allow-unsupported-compiler -arch=sm_80 -lineinfo -res-usage -src-in-ptx problem4_3d_shared.cu -o prob4.out`
Run Command: `./prob4.out`

Screenshots for each of these cases have been attached as references.